Comparing Solutions for the Two-Echelon Electric Vehicle Routing Problem

Maximus Jessey Preston Page Will Hemphill Braylon Trail

Tennessee Technological University

https://github.com/mjessey/2E-EVRP_Clarke-Wright_vs_EA

Abstract

The increased scale of delivery services has created a greater need for efficient algorithms that can find routes that minimize costs while maintaining promised time windows. In this paper, we present an in-depth analysis of solutions to the Two-Echelon Electric Vehicle Routing Problem (2E-EVRP) - an extension of the classic Vehicle Routing Problem (VRP) that allows for multiple vehicle types for long-distance and short-distance logistics.

Introduction

Over the past decade, the growth of online delivery services has led to increased demand for fast and reliable service for customers and cost-efficient solutions for companies. For example, from 2015 to 2024, Amazon has seen a 105x increase in package volume, with much of this growth occurring during the Coronavirus pandemic [1]. With both increased consumer demand and the number of Amazon warehouses, Amazon and companies like it that manage vast logistics networks like FedEx and UPS need efficient ways to be able to transport these large volumes of packages in a cheap and timely matter. This problem, known as the Vehicle Routing Problem (VRP) [4] and its extensions, is at the core of logistics networks, and thus forms a backbone for modern large-scale economics.

In this paper, we focus on one variant of the VRP specifically: the Two-Echelon Electric Vehicle Routing Problem (2E-EVRP) [5], which looks at both local and long-distance delivery and introduces a number of additional real-world constraints. We evaluate three algorithms for tackling this problem on varying input sizes and analyze their effectiveness, both in final solutions and in algorithmic efficiency. We benchmark the Clarke-Wright Savings Algorithm [6], Adaptive Large Neighborhood Search (ALNS) [7], and a Memetic approach [8].

We primarily seek to answer the following questions:

1. Which algorithm scales best with an increasing number of customers?
2. Which algorithm benefits most from additional compute? (ALNS and Memetic only)
3. How does the runtime change for different customer spatial distributions?

Answers to these questions can be found in the Conclusion section of this paper.

Background

A popular combinatorial optimization problem is the Travelling Salesman Problem (TSP) [2][3], which asks the question: "What is the optimal route for a traveling salesman to pass through each city?". More formally, given a complete weighted graph, we want to find a Hamiltonian cycle of minimal total weight. The Vehicle Routing Problem (VRP) [4] generalizes the TSP by allowing for any number of travelers as opposed to just one. It instead asks the question: "What is the optimal route for a fleet of vehicles to make a delivery to each customer?". Formally, given a similar graph along with a depot node and a fleet of k vehicles, we want to find a set of k cycles which begin and end at the depot, resulting in each customer being visited exactly once. The problem we tackle in this paper is the Two-Echelon Electric Vehicle Routing Problem [5], which extends the VRP by using two vehicle types instead of just one, each suited for different distances: larger vehicles (LVs) that transport goods in bulk from a central depot to intermediate facilities called hubs or satellites, and smaller electric vehicles (EVs) that pickup goods from the hubs and deliver them to customers. The 2E-EVRP also includes fuel constraints on electric vehicles and adds charging stations, and it enforces strict time windows in which customers must receive their goods, meaning that routes must be planned that take into account these constraints. The 2E-EVRP also allows locations such as customers, charging ports, or hubs to be visited multiple times by EVs. All these problems (the TSP, the VRP, and the 2E-EVRP) are part of the class of NP-hard problems, meaning that there is no known algorithm that can find an optimal solution in polynomial time as a function of input size.

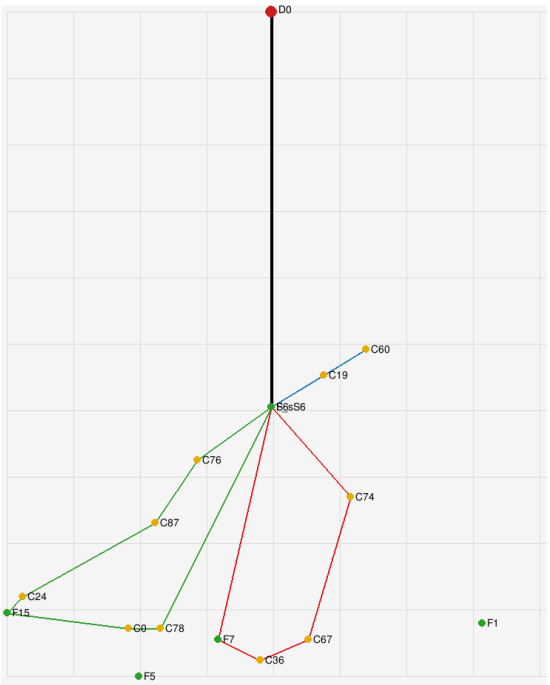

Figure 1. A visualization of a solution to the 2E-EVRP with 10 customers (clustered).

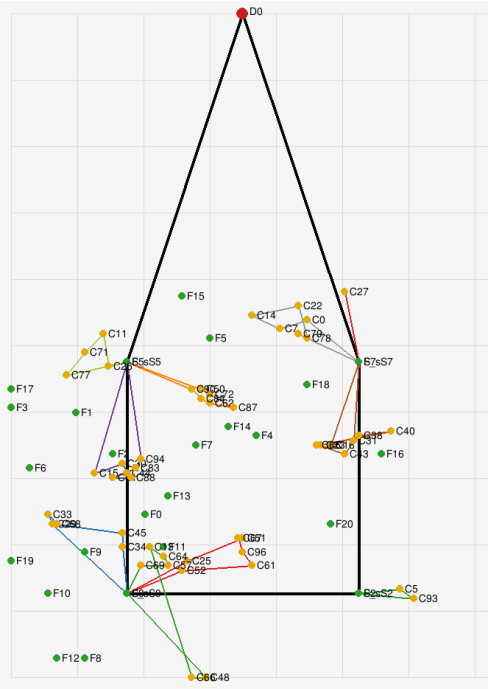

Figure 2. A visualization of a solution to the 2E-EVRP with 50 customers (clustered).

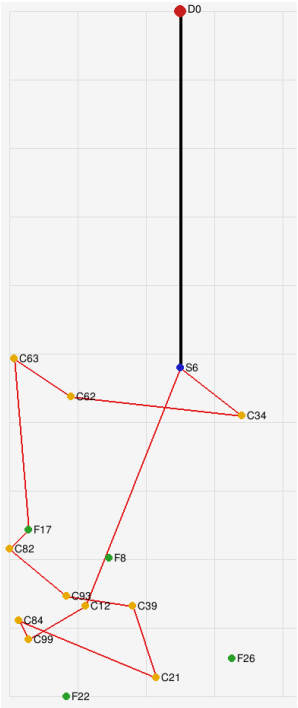

Figure 3. A visualization of a solution to the 2E-EVRP with 10 customers (random).

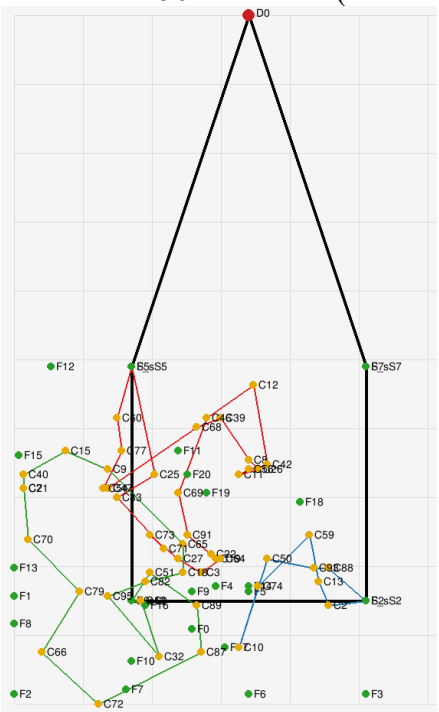

Figure 4. A visualization of a solution to the 2E-EVRP with 50 customers (random).

Figures 1-4: Solutions show vehicle routes. Black lines are for LVs, and colored lines are each for different EVs. Yellow nodes are customers, and green nodes are charging stations.

Experimental Setup

The code used in our experiments is available at the GitHub repository linked at the top of this paper. Even though C/C++ would have been better from a performance standpoint, we opted to use Python due to its large ecosystem and ease of rapid prototyping and debugging. The use of Python gave us access to libraries like PyGame for interactive visualization, Matplotlib for graphing, Multiprocessing for easy parallelization, and the potential to use Jupyter notebooks (though we ended up not using them for our experiments). We used a benchmark dataset specifically made for the 2E-EVRP by manilakbay on GitHub. The part of the dataset we used provided 1500 instances, each with their own customer, depot, and charging station positions, vehicle details (ex. max capacity and refueling rate), time windows, and customer spatial distribution (clustered, random, mixed). Additional details regarding the dataset can be found near the end of this paper. While these algorithms run just fine on typical consumer hardware, for faster experimentation, we opted to use a single compute node on Tennessee Tech's Impulse cluster, which provided us with access to 28 CPU cores. This allowed us to parallelize both benchmarking and the algorithms themselves, letting us explore more ideas than if we were doing everything sequentially.

The Clarke-Wright Savings Algorithm

This Clarke-Wright Savings Algorithm [6] is a classic approach to solving the VRP first proposed in 1964. It attempts to find a route that minimizes a given cost function by first starting with a bunch of naive/direct (and inefficient) "spoke" routes, which consist of two depots with one customer connecting them (depot -> customer -> depot), and iteratively combining them by computing each possible joint pair of customers (depot -> customer 1 -> customer 2 -> depot), calculating the "savings" (the difference between the current cost and the new cost formed by removing the trips back to the depot between customers), sorting the list of all possible savings, and merging routes in order of highest to lowest savings. This is then greedily maximized until no more routes can be combined without increasing costs. We apply this to the 2E-EVRP by introducing additional constraints for the cost function and applying the algorithm twice: once to LVs and once to EVs (which must instead travel satellite -> customer -> satellite). We hypothesize in this paper that this greedy approach makes Clarke-Wright more susceptible to getting stuck in a local optimum, and we demonstrate this in our results.

Adaptive Large Neighborhood Search

As discussed previously, the problem with Clarke-Wright is that it is a greedy solution method, and we hypothesized that this would likely cause it to more easily get trapped in a local optimum. One way to resolve this is to add noise or stochasticity. For most problems, Adaptive Large Neighborhood Search (ALNS) [7] initially starts with a random solution, and then iteratively attempts to improve it by randomly destroying parts of it and then attempting multiple ways of reconstructing it, selecting the most promising solution and repeating the process. This ensures that many solutions are tried while still being heuristically guided. This approach was first applied to a similar version of the VRP that we discuss in this paper back in 2006, and since then there have been many enhancements made that we do not discuss here. The version of ALNS that we apply to the 2E-EVRP starts with a Clarke-Wright solution rather than a random one, and if the final solution ends up being worse than the initial configuration, it falls back on Clarke-Wright. This ensures that ALNS is always at least as good as Clarke-Wright.

To increase the odds that local minima can be escaped given enough time, ALNS will sometimes select seemingly suboptimal solutions at the end of each iteration of destruction and repeated reconstruction according to a temperature parameter that slowly decreases and becomes more deterministic as the search time grows. Furthermore, destroy and repair operations are heuristically guided based on adaptive weights that are updated according to historical performance.

Unlike Clarke-Wright, ALNS can be parallelized in multiple different ways. One way to parallelize the algorithm is by attempting more repair candidate solutions in parallel. We opted to instead use an alternative approach: run ALNS multiple times in parallel and then pick the run that yields the best solution. This works because ALNS is inherently stochastic, meaning that different runs can yield different results. This does not work for Clarke-Wright.

A Memetic Approach

In general, a memetic algorithm [8] implements global search using populations (for us this will be routes) which are iterated evolutionarily, while implementing local search through refinements using a pre-determined "meme" (for us this will be based on constraints such as load capacity, battery life, etc.). The traits acquired through these refinements are then evaluated for fitness, with the traits of promising offspring being used to update the chromosomes for the next generation.

Our implementation of the memetic algorithm uses a multi-step process, starting by first creating a series of base line routes to work with using a pseudo-greedy approach, then utilizing ALNS for the local search on each individual route, to further refine the starting population. From there, we enter our main loop, which consists of randomly selecting two parents from the population, generating offspring using our backbone crossover class, using ALNS to improve the offspring, and updating the global best if a produced offspring is a new best route. The offspring is then inserted into the population by calculating a “goodness score” based on its solution quality and

population diversity. Finally, once the stopping criterion has been reached (in our case, this is a limit on time), the best solution is returned.

We parallelize this algorithm in the same way as we did for ALNS.

Results

We first compared Clarke-Wright, ALNS, and memetic in three ways for an increasing number of customers: average wall-clock time, average route distance, and average number of EVs used.

We measured two ways of increasing available compute for ALNS and the memetic algorithm: increasing maximum wall-clock time, and CPU core parallelization.

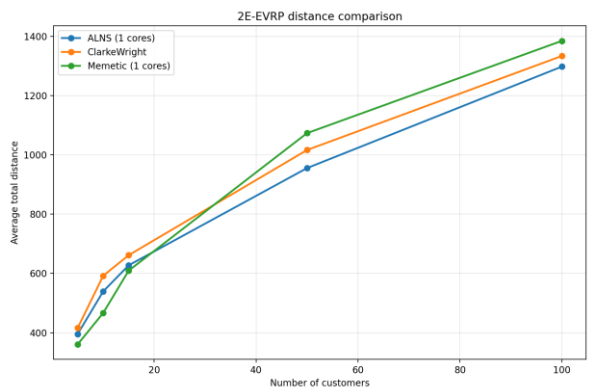

Figure 5. ALNS finds the shortest routes as the number of customers increases while Memetic performs the worst

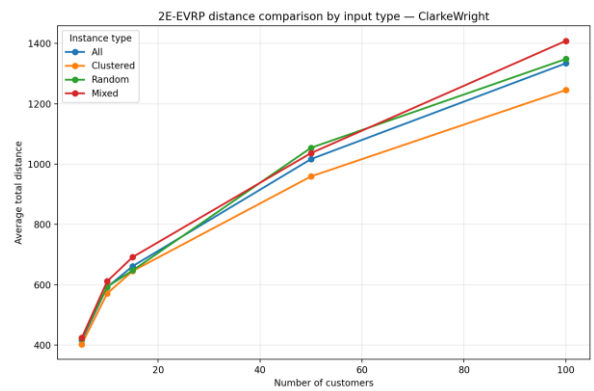

Figure 6. Clustered customers allow for shorter routes on average compared to randomly scattered customers.

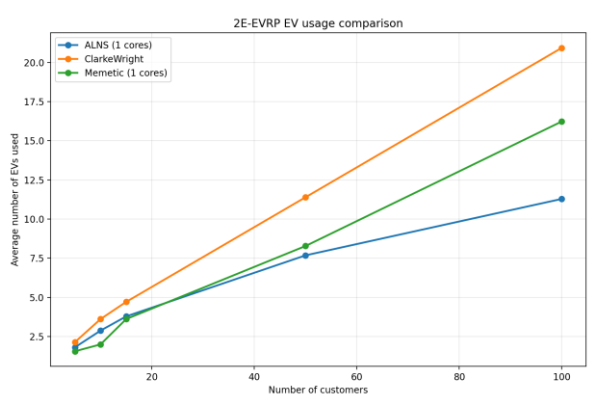

Figure 7. The number of EVs needed appears to grow linearly for Clarke-Wright and Memetic, but logarithmically for ALNS.

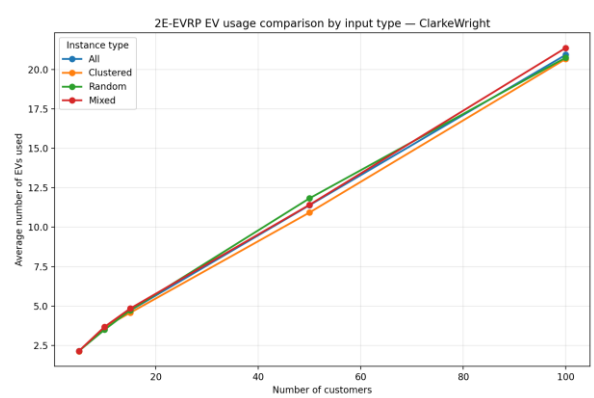

Figure 8. The number of EVs required does not depend on instance type. This is true for all algorithms.

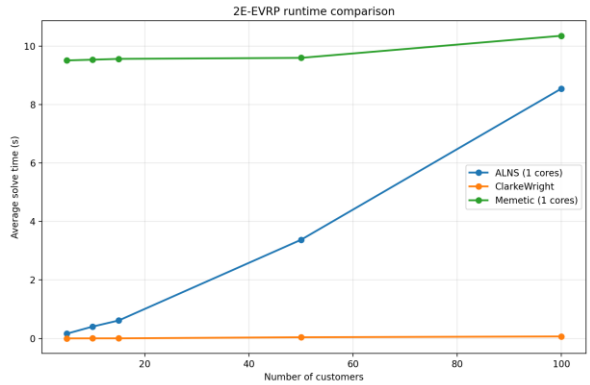

Figure 9. ALNS shows the highest runtime complexity with increasing input size.

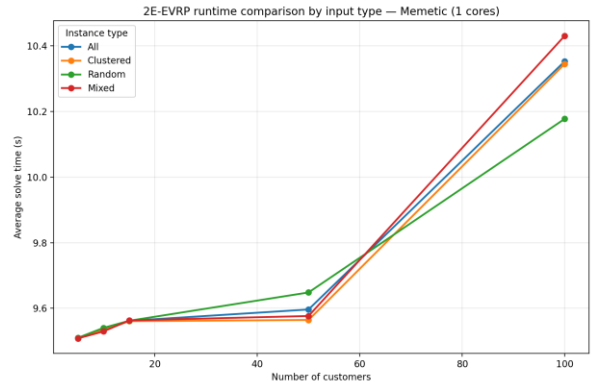

Figure 10. Despite Memetic having strict time limits, there is a sudden jump past 50 customers.

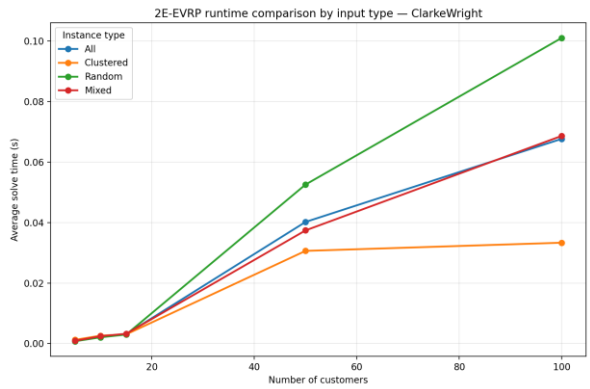

Figure 11. Runtime jumps up suddenly between 15 and 50 customers for Clarke-Wright and then appears to start to flatten off.

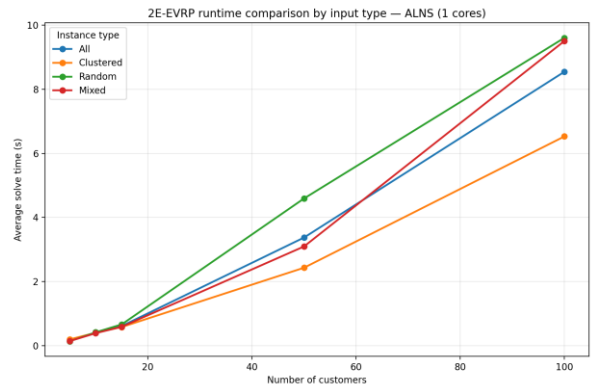

Figure 12. Runtime appears to increase quadratically for ALNS.

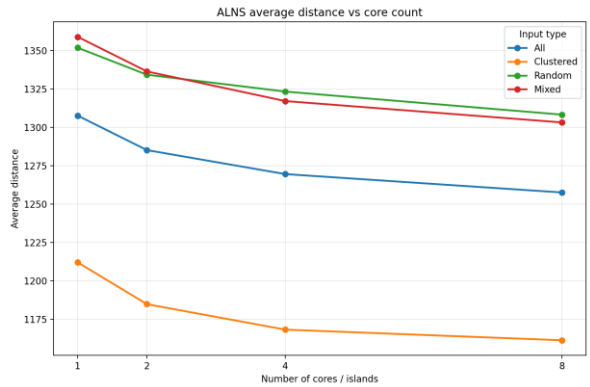

Figure 13. ALNS is able to find shorter routes given more CPU cores.

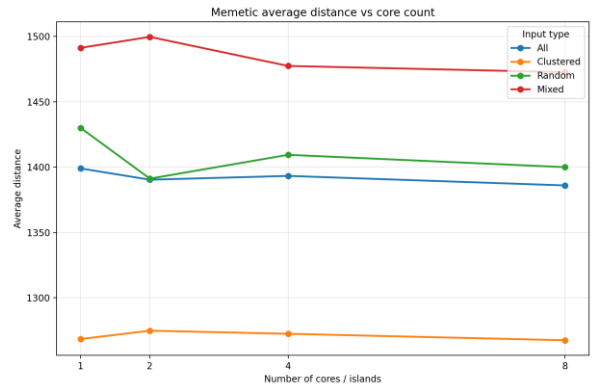

Figure 14. Memetic does not appear to benefit from increased CPU cores.

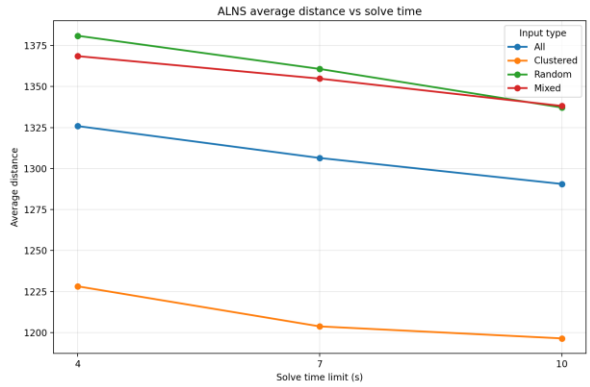

Figure 15. ALNS is able to find shorter routes given more processing time.

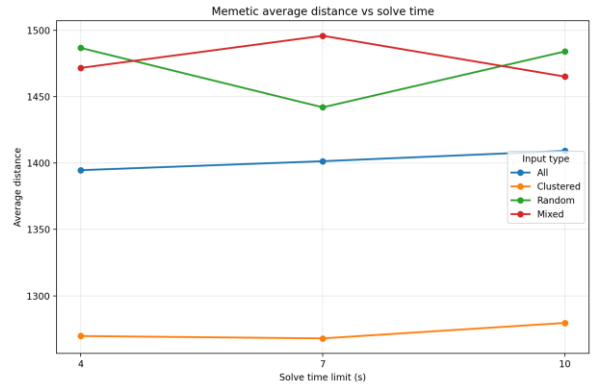

Figure 16. Memetic does not appear to benefit much from increased processing time.

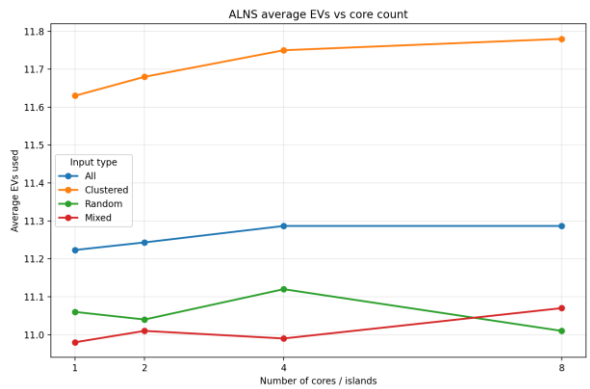

Figure 17. ALNS does not appear to find solutions that use fewer EVs when given more CPU cores.

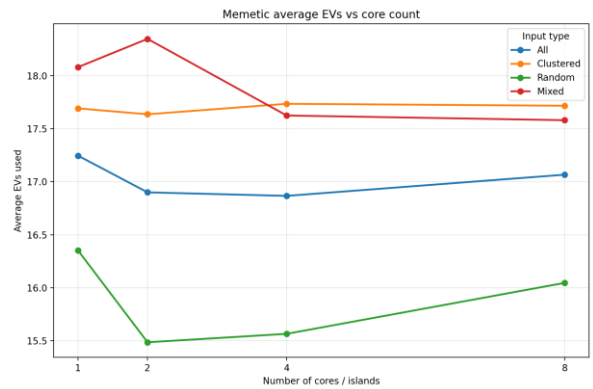

Figure 18. Memetic does not appear to find significantly better solutions that use fewer EVs when given more CPU cores.

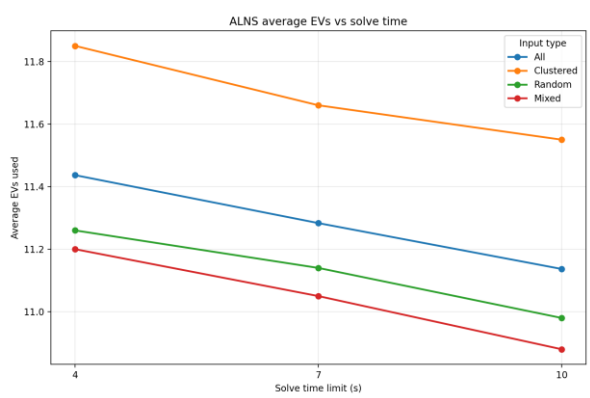

Figure 19. ALNS is able to find solutions that use fewer EVs when given more processing time.

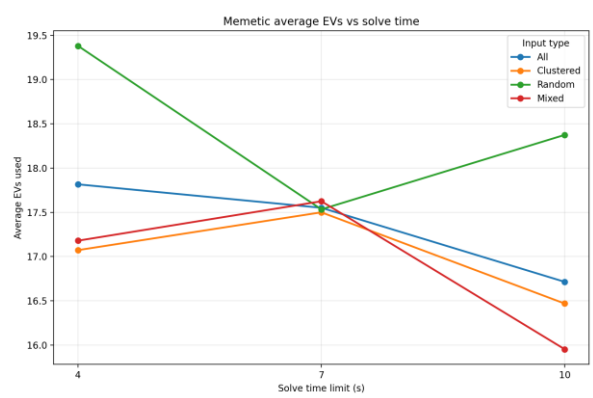

Figure 20. Memetic appears to only be able to find solutions that use fewer EVs when given more processing time when the customers are not randomly distributed (i.e. when they are clustered).

Conclusion

Using the graphs from the results section, we are now able to answer our three questions and expand upon them with further investigation:

1. **"Which algorithm scales best with an increasing number of customers?"**
   - **"How does runtime scale?"** ALNS and Memetic are both "any-time" algorithms, meaning that they can provide a solution under any fixed time constraints. This means that runtime for these algorithms as inputs scale should in theory be constant. However, only Memetic was tested with fixed time limits, while ALNS instead had a fixed iteration limit. In retrospect, it probably would have been a better comparison had both used fixed time limits. With fixed iteration limits though, ALNS appears to scale quadratically with input size, which matches with our theoretical time complexity of $O(n^2)$. For Clarke-Wright however, we do not seem to observe a $O(n^2 \log n)$ time complexity as predicted. Instead, Clarke-Wright seems to scale either linearly or logarithmicly with input size, but due to insufficient data, it is hard to draw any conclusions.
   - **"How does travel distance scale?"** ALNS consistently beats Clarke-Wright, which proves our hypothesis correct that Clarke-Wright gets stuck in a local optimum, as stochasticity can be used to escape it. It also aligns with our expectation that ALNS should always perform at least as well as Clarke-Wright. We see that Memetic is beaten by both Clarke-Wright and ALNS, however, this does not necessarily mean that Memetic's solutions are worse, as this doesn't take into account EV usage.
   - **"How does EV usage scale?"** Clarke-Wright uses the most EVs, while Memetic and ALNS use less. However, it looks like the number of EVs that both Clarke-Wright and Memetic use scales linearly with input size while for ALNS this is logarithmic. A larger input size is needed to see if this trend continues.
2. **"Which algorithm benefits most from additional compute?"** Surprisingly, Memetic did not seem to benefit significantly from additional compute (whether that be more processing time or more resources in the form of cores) as ALNS. This shows that ALNS scales better with more compute than the other two algorithms.
3. **"How does runtime change for different customer spatial distributions?"** It is faster to solve instances when customers are clustered vs randomly distributed for all algorithms except Memetic (the reason why Memetic is different is because it used fixed time constraints). This matters because customer distribution can vary based on whether customers are in an urban (where there is more clustering) vs a rural area (where there is less clustering). We believe the reason for this increased solve time for randomly distributed customers is because these instances also have less constraints on fuel and time windows, meaning there are more possible feasible solutions that must be searched.

Based on our results, we would recommend that companies go with using Adaptive Large Neighborhood Search (ALNS) for solving the 2E-EVRP due to the following reasons:

1. ALNS is guaranteed to be at least as good as Clarke-Wright.
2. EV usages scales very efficiently with ALNS for large numbers of customers.
3. ALNS scales well with additional compute (especially additional runtime).

References

1. Amazon Logistics Statistics. Capital One Shopping Research. 2026.
2. History of the TSP. University of Waterloo. 2022.
3. Didier Maquin. The Traveling Salesman Problem. CRAN. 2025. ⟨hal-04909391v2⟩
4. G. B. Dantzig, J. H. Ramser, (1959) The Truck Dispatching Problem. Management Science 6(1):80-91. https://doi.org/10.1287/mnsc.6.1.80
5. Ulrich Breunig, Roberto Baldacci, Richard F. Hartl, Thibaut Vidal. The Electric Two-echelon Vehicle Routing Problem. ScienceDirect. 2018. https://doi.org/10.1016/j.cor.2018.11.005
6. G. Clarke, J. W. Wright, (1964) Scheduling of Vehicles from a Central Depot to a Number of Delivery Points. Operations Research 12(4):568-581. https://doi.org/10.1287/opre.12.4.568
7. Stefan Ropke, David Pisinger, (2006) An Adaptive Large Neighborhood Search Heuristic for the Pickup and Delivery Problem with Time Windows. Transportation Science 40(4):455-472. https://doi.org/10.1287/trsc.1050.0135
8. Peng B, Zhang Y, Gajpal Y, Chen X. A Memetic Algorithm for the Green Vehicle Routing Problem. Sustainability. 2019; 11(21):6055. https://doi.org/10.3390/su11216055

Dataset Details

The dataset used contains five input sizes (measured in number of customers): 5, 10, 15, 50, and 100. For each of these input sizes, there are 300 instances, for a total of 1500 instances in the whole dataset. For every input size, the instances are divided into three groups for the distribution of customer positions: random, clustered, and mixed. Each instance contains the following information:

1. Node Information
   i. Type: Depot, satellite (hub), charging station, or customer
   ii. Coordinates: x and y Position
   iii. Delivery Demand: Amount to deliver to the customer
   iv. Pickup Demand: Amount to pick up from the customer

   v. Time Windows: Ready time and due date
   vi. Service Time: Time required to service the node (whether that be charging, dropping off goods, or picking up goods)
2. Vehicle Information
   i. LV Loading Capacity
   ii. EV Loading Capacity
   iii. EV Battery Capacity
   iv. Fuel Consumption Rate
   v. Average Velocity

Further Research

We suspect that there are better ways to parallelize ALNS and Memetic beyond what we did in our experiments. For example, instead of running the same algorithm in parallel and independently and then selecting the best result, we could have instead parallelized the internal steps of the algorithm. For ALNS, this would be parallelizing the destroy-and-repair operations, and for Memetic, that would be evaluating individuals within a population in parallel.

Further research also needs to be done exploring a wider range of input sizes. The dataset we used provided only five input sizes: 5, 10, 15, 50, and 100. These input sizes however are small in comparison to the large number of customers that must be serviced in the real world, so a dataset that contains larger input sizes would be preferable.

Likewise, further research needs to be done exploring a wider range of computational constraints. We were only able to benchmark a few different core counts (1, 2, 4, 8) and a few different time constraints (4, 7, and 10 seconds), but we imagine that more information could be acquired from further testing with a wider range of constraints.

AI Disclosure

Generative AI in the form of large language models (LLMs) were used to assist in writing code, converting code to pseudocode, and helping us understand the algorithms used. The models used were o3, GPT-5.4 and GPT-5.5, and Claude Sonnet 4.6.

Algorithm Pseudocode

Clarke-Wright Savings Algorithm

```
Algorithm: Clarke–Wright Savings Heuristic for the 2E-EVRP
```

```
 Input:
    Depot d0, satellites S, customers C, charging stations F,
    EV capacity CEV, LV capacity CLV, battery capacity Q

Output:
    First-echelon LV routes and second-echelon EV routes

1:  Assign each customer to its nearest satellite

2:  for each satellite s ∈ S:
3:      Initialize EV routes Rs ← {[s, i, s] : customer i assigned to s}
4:      Compute savings σij = d(s,i) + d(s,j) – d(i,j) for all customer pairs i,j
5:      Sort savings in decreasing order

6:      for each pair (i,j) in sorted savings:
7:          Let ri and rj be the routes containing i and j
8:          if ri ≠ rj and i,j are mergeable route endpoints:
9:              r ← merge ri and rj by replacing edges (i,s) and (s,j) with (i,j)
10:             if load(r) ≤ CEV:
11:                 Insert charging stations if needed
12:                 if r satisfies battery and time-window constraints:
13:                     Replace ri and rj with merged route r

14:     Repair any remaining battery-infeasible EV routes by inserting charging
stations
15:     Store Rs as the EV routes for satellite s

16: Compute demand q_s served through each satellite s

17: Initialize LV routes RLV ← {[d0, s, d0] : q_s > 0}

18: Compute savings σij = d(d0,i) + d(d0,j) – d(i,j) for all active satellite pairs
i,j
19: Sort savings in decreasing order

20: for each pair (i,j) in sorted savings:
21:     Let ri and rj be the LV routes containing satellites i and j
22:     if ri ≠ rj and i,j are mergeable route endpoints:
23:         r ← merge ri and rj by replacing edges (i,d0) and (d0,j) with (i,j)
24:         if load(r) ≤ CLV:
25:             Replace ri and rj with merged route r

26: Return LV routes RLV and EV routes {Rs}
```

Adaptive Large Neighborhood Search

```
Algorithm: ALNS for the 2E-EVRP

Input:
    Instance data, time/iteration limit, EV and LV capacities,
    battery capacity, customer time windows

Output:
    Best feasible 2E-EVRP solution
```

```
xcur ← ClarkeWright(instance)
xbest ← xcur
Initialize weights for all destroy and repair operators

Destroy operators:
    random removal,
    worst-cost removal,
    route removal,
    tight time-window removal,
    geographic cluster removal

Repair operators:
    greedy insertion,
    regret insertion

T ← initial temperature

while stopping condition is not met:
    Choose one destroy operator d using adaptive weights
    Choose one repair operator r using adaptive weights

    xdestroyed, removed ← d(xcur)
    x’ ← r(xdestroyed, removed)

    if x’ is feasible:
        Δ ← cost(x’) − cost(xcur)

        if cost(x’) < cost(xbest):
            xbest ← x’
            reward selected operators with best-solution score

        if Δ < 0:
            xcur ← x’
            reward selected operators with improvement score

        else if random(0,1) < exp(−Δ / T):
            xcur ← x’
            reward selected operators with accepted-solution score

        else:
            reward selected operators with rejection score

    T ← cooling_rate · T

    Periodically update operator weights:
        new_weight ← (1 − ρ) · old_weight + ρ · average_score

Return xbest
```

Repair Procedure

```
for each removed customer i:
    Evaluate all feasible insertion positions in existing EV routes
```

```
    Keep only insertions satisfying EV capacity, battery, and time windows
    Insert i in the position with minimum added distance
    If no feasible position exists, create a new route from the nearest satellite
Return repaired solution if globally feasible
```

Memetic Algorithm

```
Algorithm: Memetic Algorithm for the 2E-EVRP

Input:
    2E-EVRP instance, population size np, time limit Tmax,
    k-greedy parameter k, diversity weight δ

Output:
    Best solution found xbest

P ← GenerateInitialPopulation(np, k)
for each solution x ∈ P:
    x ← AdaptiveLocalSearch(x)

xbest ← best solution in P

while time limit Tmax is not reached:
    Randomly select two parents xa, xb ∈ P

    xchild ← BackboneCrossover(xa, xb)

    if xchild is infeasible or empty:
        continue

    xchild ← AdaptiveLocalSearch(xchild)
    Optionally improve xchild by eliminating a small EV route and reinserting its customers

    if cost(xchild) < cost(xbest):
        xbest ← xchild

    P ← LCSPopulationUpdate(P, xchild, δ)

Return xbest


Procedure GenerateInitialPopulation(np, k)

P ← ∅
Generate one nearest-neighbor solution using k = 1 and add it to P

while |P| < np:
    Assign each customer to its nearest satellite
    for each satellite s:
        Build EV routes using k-Pseudo Greedy:
            Start a route at s
```

```
                Repeatedly select one of the k nearest feasible unvisited customers
                Feasibility checks include EV capacity, battery, time windows,
                and ability to return to s directly or through a charging station
                Insert charging stations when needed
                Close the route at s when no feasible customer can be added

        Build LV routes using Clarke-Wright savings on satellite demands
        Add the constructed solution to P

    Return P


Procedure AdaptiveLocalSearch(x)

    Initialize scores for eight neighborhood moves
    xcur ← x, xbest ← x

    while stopping criterion is not met:
        Select a neighborhood move by roulette-wheel selection

        Generate the best feasible neighbor x’ using the selected move
        Neighborhoods include intra-route and inter-route:
        customer insertion, customer swap, arc insertion, and arc swap

        if cost(x’) < cost(xcur):
            xcur ← x’
            Reward the selected move

            if cost(x’) < cost(xbest):
                xbest ← x’

        else:
            Penalize the selected move

        If the search is stuck, perturb xbest by removing customers
        and reinserting them greedily

    Return xbest


Procedure BackboneCrossover(xa, xb)

    U ← set of all customers
    Create route pools from the EV routes of parents xa and xb
    xchild ← empty solution

    while U is not empty:
        Collect parent routes containing at least one customer in U

        if no such route exists:
            Greedily insert remaining customers in U
            break

        For each candidate route r:
            θ ← number of unassigned customers in r
```

```
        score(r) ← route_distance(r) / θ

    Select the route r* with minimum score
    Remove already assigned customers from r*
    Add the cleaned route r* to xchild
    Remove customers of r* from U
    Remove those customers from all remaining parent routes

Build LV routes for xchild using Clarke-Wright savings

if xchild is infeasible:
    Repair infeasible EV routes by splitting them into single-customer routes

Return xchild


Procedure LCSPopulationUpdate(P, xchild, δ)

P+ ← P ∪ {xchild}
Flatten each solution into a sequence of customers and charging stations

for each solution x ∈ P+:
    Compute its minimum LCS distance to every other solution

for each solution x ∈ P+:
    quality(x) ← normalized inverse cost
    diversity(x) ← normalized minimum LCS distance
    goodness(x) ← δ · quality(x) + (1 − δ) · diversity(x)

Let xworst be the solution in P with the smallest goodness score

if goodness(xchild) > goodness(xworst):
    Replace xworst with xchild

Return P
```